SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	P. MUNI KARTHIK	**Reg. No.:**	192425061
Date:		**Duration:**	60 minutes

Code of Conduct

I P. MUNI KARTHIK / 192425061 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. MUNI KARTHIK

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design** ○ Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization** ○ Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy** ○ Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.

5. **Version Control Workflow** ○ Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
    - Explain the purpose of each Git operation performed.
6. **Commit History Analysis** ○ Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design** ○ Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development** ○ Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design** ○ Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing** ○ Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker** ○ Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements** ○ Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. PROBLEM ANALYSIS

1.1 Introduction

The Data Warehouse Management System is a web-based application designed to manage warehouse-related data in a centralized system. The system allows administrators or warehouse employees to maintain information about products, warehouses, stock quantities and inventory movements.

Traditional warehouse management using spreadsheets or manually maintained records can result in data duplication, errors, difficulty in tracking stock and delays in generating reports. The proposed system provides a centralized database and web interface for managing warehouse information efficiently.

The application is developed using Python Flask for the backend, HTML and CSS for the user interface, and PostgreSQL for data storage. Docker is used to create a consistent execution environment, while Git is used for version control and collaborative development.

1.2 Problem Statement

Organizations managing multiple warehouses need an efficient system to maintain product and inventory information. Manual methods make it difficult to determine available stock, identify low-stock products and obtain a consolidated view of warehouse operations.

The Data Warehouse Management System solves this problem by providing a centralized application that stores warehouse information and provides users with simple operations for managing and monitoring inventory data.

1.3 Project Objectives

The main objectives are:

- To develop a centralized warehouse management application.
- To store product and warehouse information systematically.
- To maintain stock quantities accurately.
- To provide a simple web-based interface.
- To generate warehouse and inventory summaries.
- To use PostgreSQL as the database.
- To use Git for version control.
- To use Docker for application containerization.
- To provide a reproducible development and deployment environment.

1.4 Functional Requirements

The system provides the following functional requirements:

1. Add product details.
2. View available products.
3. Add warehouse information.
4. View warehouse information.
5. Record stock quantities.
6. Display inventory information.
7. Identify products with low stock.
8. Display summary information through a dashboard.
9. Store data permanently in PostgreSQL.
10. Run the complete application using Docker containers.

1.5 Non-Functional Requirements

The system should satisfy the following non-functional requirements:

- **Performance:** The system should respond quickly to normal user requests.
- **Reliability:** Data should be stored consistently in the database.
- **Maintainability:** The source code should be organized into separate components.
- **Portability:** The application should run on different systems using Docker.
- **Scalability:** Additional services can be added in the future.
- **Security:** Database credentials should be managed using environment variables.
- **Usability:** The interface should be simple and easy to understand.

1.6 Expected Outcome

The expected outcome is a containerized web application capable of managing warehouse and inventory information. The project should be stored in a Git repository with meaningful commits and branches. The final application should be accessible through a web browser after starting the Docker Compose environment.

2. PROJECT STRUCTURE DESIGN

2.1 Repository Structure

The proposed repository structure is:

```
data-warehouse-management/
│
├── app.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .gitignore
├── README.md
│
├── templates/
│   └── index.html
│
├── static/
│   └── style.css
│
└── database/
    └── init.sql
```

2.2 Purpose of Files and Folders

app.py

This is the main Flask application file. It contains the backend routes and database operations.

requirements.txt

This file contains the Python packages required by the application.

Dockerfile

The Dockerfile contains instructions for building the Docker image for the Flask application.

docker-compose.yml

Docker Compose defines and manages the application and PostgreSQL database services.

.gitignore

This file specifies files and folders that should not be tracked by Git.

README.md

This file provides information about the project, installation, execution and usage.

templates/

This folder contains HTML templates used by Flask.

static/

This folder contains CSS files used to design the web interface.

database/

This folder contains SQL scripts used to initialize the PostgreSQL database.

---

3. GIT REPOSITORY INITIALIZATION

3.1 Git

Git is a distributed version control system used to track changes in source code. It allows developers to create versions of a project, work on separate branches and merge changes.

Git is suitable for this project because the application contains multiple source files and may be developed or modified over several iterations.

3.2 Repository Initialization

The repository can be initialized using:

mkdir data-warehouse-management

cd data-warehouse-management

git init

The git init command creates a new Git repository in the project directory.

3.3 Adding Project Files

After creating the project files:

git status

git add .

git commit -m "Initial project setup"

The git status command displays changed and untracked files.

The git add . command stages the project files.

The git commit command permanently records the staged changes in the local repository.

3.4 Essential Git Files

The main Git-related directory is:

.git/

The .git directory contains repository metadata, commit information, references, objects and configuration.

The .gitignore file is used to prevent unnecessary files from being tracked.

Example:

__pycache__/

*.pyc

.env

venv/

.git/

---

4. GIT BRANCHING STRATEGY

4.1 Selected Strategy

The project uses a simplified Git Flow strategy:

main

```
  │
  └── develop
      │
      ├── feature/database
      ├── feature/backend
      ├── feature/frontend
      └── feature/docker
```

4.2 Main Branch

The main branch contains stable versions of the application.

Only tested and working code should be merged into the main branch.

4.3 Develop Branch

The develop branch contains the latest development version before release.

Developers can integrate feature branches into this branch.

4.4 Feature Branches

Separate feature branches are created for individual tasks.

Examples:

git checkout -b feature/database

git checkout -b feature/backend

git checkout -b feature/frontend

git checkout -b feature/docker

Each feature can be developed independently.

4.5 Why This Strategy Is Appropriate

The selected branching model provides:

- Separation of development and stable code.
- Independent development of features.
- Easier debugging.
- Better collaboration.
- Controlled integration.
- Reduced risk of breaking the main application.

---

5. VERSION CONTROL WORKFLOW

5.1 Basic Workflow

The Git workflow followed by the project is:

Create Feature

↓

Create Branch

↓

Write/Modify Code

↓

git status

↓

git add

↓

git commit

↓

git push

↓

Pull Request / Merge

↓

Develop

↓

Main

5.2 Creating a Feature Branch

git checkout develop

git checkout -b feature/backend

This creates an independent branch for backend development.

5.3 Committing Changes

git add .

git commit -m "Add Flask backend"

The commit stores the completed change in the Git history.

5.4 Merging

After testing the feature:

git checkout develop

git merge feature/backend

The feature branch is merged into the development branch.

5.5 Synchronizing Repository

For a GitHub repository:

git remote add origin <repository-url>

git push -u origin main

git push origin develop

To obtain changes from the remote repository:

git pull origin develop

5.6 Conflict Resolution

If two branches modify the same section of a file, Git may report a merge conflict.

The conflict is resolved manually by editing the affected file.

After resolving:

git add .

git commit -m "Resolve merge conflict"

This records the conflict resolution.

---

6. COMMIT HISTORY ANALYSIS

Meaningful commit messages should describe the change performed.

Example commit history:

a91d2f1 Add Docker configuration

c83b7a2 Add inventory dashboard

b72ac91 Add Flask backend routes

a41dc72 Add PostgreSQL database schema

91fc382 Add frontend interface

72ab101 Add project structure

6.1 Good Commit Practices

The project follows these practices:

- Each commit represents a meaningful change.
- Commit messages are short and descriptive.
- Unrelated changes are avoided in the same commit.
- Feature development is performed using branches.
- Stable code is merged into the main branch only after testing.

For example:

Good:

Add PostgreSQL inventory schema

Good:

Implement inventory dashboard

Poor:

changes

Poor:

update

Poor:

final code

Meaningful commit messages make it easier to understand project development and identify when a particular feature was introduced.

---

7. DOCKER ENVIRONMENT DESIGN

7.1 Introduction to Docker

Docker is a containerization platform that packages an application together with its dependencies into isolated containers.

The application can therefore run consistently on different systems without requiring developers to manually configure all dependencies.

7.2 Why Docker Is Suitable

Docker is suitable for this project because the system requires:

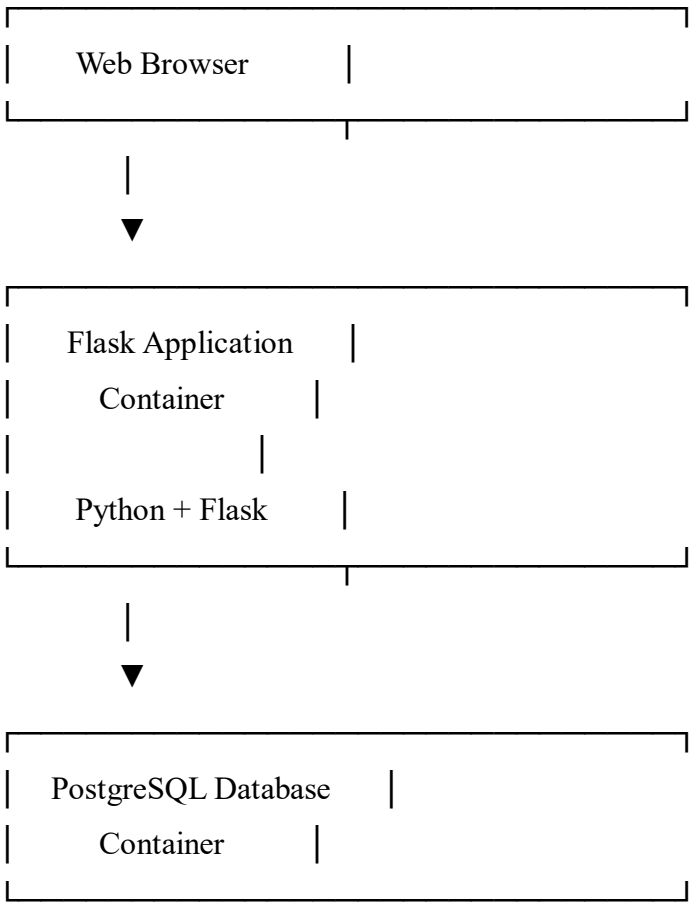
- Python.
- Flask.
- PostgreSQL.
- Python dependencies.
- Database configuration.

Without Docker, different developers may have different Python or PostgreSQL configurations.

With Docker, the required environment is defined using configuration files.

7.3 Application Components

The system contains two main containers:



The Flask container handles application requests, while PostgreSQL stores the data.

---

8. DOCKERFILE DEVELOPMENT

8.1 Dockerfile

The following Dockerfile is used:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```

EXPOSE 5000

CMD ["python", "app.py"]

8.2 Explanation of Instructions

FROM

FROM python:3.11-slim

Provides a lightweight Python environment as the base image.

WORKDIR

WORKDIR /app

Sets /app as the working directory inside the container.

COPY

COPY requirements.txt .

Copies the dependency file into the container.

RUN

RUN pip install –no-cache-dir -r requirements.txt

Installs the required Python packages.

COPY

COPY . .

Copies the application source code into the container.

EXPOSE

EXPOSE 5000

Documents the port used by the Flask application.

CMD

CMD [“python”, “app.py”]

Starts the Flask application when the container runs.

---

9. DOCKER COMPOSE DESIGN

9.1 Docker Compose

Docker Compose is used to define and manage multiple containers using a single configuration file.

The project contains two services:

1. Flask application
2. PostgreSQL database

9.2 docker-compose.yml

```
services:

  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      DB_HOST: db
      DB_NAME: warehouse
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: warehouse
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
```

volumes:

postgres_data:

9.3 Services

web

The web service builds the Flask application from the Dockerfile.

db

The db service uses the official PostgreSQL Docker image.

9.4 Networking

Docker Compose automatically creates a network for the services.

The Flask application can access PostgreSQL using:

db

instead of localhost.

9.5 Volumes

The PostgreSQL volume:

postgres_data:/var/lib/postgresql/data

provides persistent storage for database information.

Even if the PostgreSQL container is recreated, the stored data can remain in the Docker volume.

9.6 Environment Variables

The database configuration is passed using environment variables:

DB_HOST

DB_NAME

DB_USER

DB_PASSWORD

This avoids hard-coding configuration values directly into the application.

9.7 Dependencies

The following configuration:

```
depends_on:
  - db
```

ensures that the database service is started before the web service.

---

10. CONTAINER DEPLOYMENT AND TESTING

10.1 Building the Application

The project can be started using:

```
docker compose build
```

This builds the Flask application image.

10.2 Starting Containers

```
docker compose up
```

The command starts the application and PostgreSQL containers.

For background execution:

```
docker compose up -d
```

10.3 Checking Running Containers

```
docker compose ps
```

The command displays the status of the services.

10.4 Viewing Logs

```
docker compose logs
```

Logs can be used to identify application errors.

For the web service:

```
docker compose logs web
```

For the database:

```
docker compose logs db
```

10.5 Application Testing

After the containers are running, the application can be accessed using:

http://localhost:5000

The following tests can be performed:

Test	Expected Result
Open application	Dashboard displayed
Add product	Product stored successfully
View products	Product information displayed
Add warehouse	Warehouse stored successfully
Add stock	Inventory quantity updated
View inventory	Current stock displayed
Restart containers	Application starts successfully
Database restart	Persistent data remains available

11. BENEFITS OF GIT AND DOCKER

11.1 Benefits of Git

Version Management

Git maintains a history of project changes and allows previous versions to be restored.

Collaboration

Multiple developers can work on different branches and merge their work.

Branching

Features can be developed independently without affecting stable code.

Traceability

Commit history makes it possible to identify who changed the code and what was changed.

Backup

A remote Git repository provides an additional copy of the project.

11.2 Benefits of Docker

Portability

The application can run consistently on different computers.

Environment Consistency

All developers use the same application environment.

Easy Deployment

The application can be started using:

docker compose up

Isolation

The Flask application and PostgreSQL database run in separate containers.

Maintainability

Dependencies and deployment configuration are defined using Docker files.

Scalability

Additional services can be added later, such as Redis, Nginx or another backend service.

---

12. CHALLENGES AND IMPROVEMENTS

12.1 Challenges

Several challenges can occur during implementation.

Git Merge Conflicts

Changes made by different developers may result in merge conflicts.

Docker Configuration

Incorrect ports, environment variables or service names can prevent containers from communicating.

Database Connectivity

The Flask application must use the Docker service name instead of localhost when connecting to PostgreSQL.

Dependency Issues

Different Python package versions can result in compatibility problems.

Data Persistence

Without a Docker volume, database data can be lost when containers are removed.

12.2 Possible Improvements

The following improvements can be implemented in the future:

- Add user authentication.
- Add role-based access control.
- Add advanced inventory reports.
- Add product search and filtering.
- Add low-stock notifications.
- Add data visualization dashboards.
- Add automated testing.

- Add CI/CD using GitHub Actions.
- Use Docker secrets for sensitive credentials.
- Add Nginx as a reverse proxy.
- Deploy the system to a cloud platform.
- Add monitoring and logging.
- Implement database backup and recovery.

CONCLUSION

The Data Warehouse Management System demonstrates the practical application of software engineering concepts including version control and containerization.

Git provides an organized mechanism for managing source code, tracking changes, creating branches and collaborating on the project. The selected branching strategy separates stable code from ongoing development and allows individual features to be developed independently.

Docker provides a consistent environment for running the Flask application and PostgreSQL database. Docker Compose simplifies the management of the multiple application services and provides networking, persistent storage and environment configuration.

By combining Git and Docker, the project becomes more portable, maintainable, reproducible and suitable for collaborative development and deployment.

The implementation therefore satisfies the CO3 objective of building applications with an emphasis on containerization, version control and collaborative software development.

REFERENCES

1. Git Documentation – Git version control concepts and commands.
2. Docker Documentation – Docker containers and images.
3. Docker Compose Documentation – Multi-container application management.
4. Python Flask Documentation – Flask web application framework.
5. PostgreSQL Documentation – PostgreSQL relational database system.
6. Software Engineering course materials – Version control and deployment concepts.